Laboratorio 9. Embeddings

Sección 1. Preparación de los corpus

Trabajé con el corpus de noticias, que después de quitar duplicados y textos vacíos quedó con 1,138 documentos, y con el Quijote. En el Quijote eliminé el encabezado y la licencia de Gutenberg junto con la tasa, las licencias, la dedicatoria y el prólogo, y tomé cada capítulo como documento porque se detectan fácilmente y son una unidad parecida a una noticia. Encontré 126 capítulos, 52 de la primera parte y 74 de la segunda. Cada corpus se segmentó en oraciones con NLTK y cada oración se tokenizó conservando el orden, de modo que el resultado es una lista de listas de tokens. Las noticias quedaron con 14,388 oraciones, 588,216 tokens y 28,461 tipos, y el Quijote con 10,391 oraciones, 368,258 tokens y 22,443 tipos.

La normalización fue pasar todo a minúsculas, quitar la puntuación, conservar las tildes y dejar los números enteros como tokens. Esta decisión salió de revisar el vocabulario de fastText SUC, que está completamente en minúsculas, distingue tildes porque españa sí está y España no, y también guarda números como 2019. Con esta normalización SUC cubre el 98.86 por ciento de los tokens de las noticias y el 97.40 por ciento de los del Quijote. A diferencia del Laboratorio 1 no quité stopwords ni lematicé. Quitar stopwords había bajado los tokens de 631,481 a 318,949, lo que cambia por completo las ventanas de contexto, y la lematización junta formas como gobiernos y gobernó, perdiendo justo la morfología que se necesita para las analogías de plural y tiempos verbales. Las columnas del Laboratorio 1 solo las usé en los baselines de BoW y TF-IDF.

Con min_count igual a 1, 2 y 5 el vocabulario de las noticias fue de 28,461, 16,131 y 8,347 palabras, y los tokens que quedan fuera fueron 0, 2.10 y 5.62 por ciento. En el Quijote fueron 22,443, 11,515 y 5,220 palabras con 0, 2.97 y 7.44 por ciento fuera. Al generar a mano los pares de skip-gram con ventana 2 para la oración nunca pensé que pudiera trabajar en un banco salieron 26 pares, y para todo el corpus salieron 2,266,536 pares con ventana 2 y 5,451,278 con ventana 5, es decir 3.85 y 9.27 pares por token. Word2vec no necesita datos etiquetados porque la etiqueta es la misma palabra vecina que ya está en el texto, por lo que es aprendizaje auto supervisado.

Sección 2. Co-ocurrencia, PPMI y SVD

La matriz de co-ocurrencia de las 2,000 palabras más frecuentes con ventana de dos palabras quedó de 2000 por 2000 con 93.67 por ciento de ceros, menos dispersa que la matriz BoW del Laboratorio 2, que tenía 99.17 por ciento. Con conteos crudos los cosenos no coincidieron con mi intuición, porque tecnología e inflación salieron con 0.9508 mientras inflación y precios solo con 0.5268. Las palabras que más co-ocurren con empresa y con gobierno fueron casi todas stopwords como la, de, el, que y en, y por eso todas las filas se parecen. Después de implementar PPMI con NumPy los pares

relacionados quedaron arriba, entre 0.19 y 0.26, y los no relacionados abajo, con 0.0666 y 0.0325. Las palabras más asociadas pasaron a ser de contenido, como telecomunicaciones y software para empresa o federal y funcionarios para gobierno, aunque también aparecen nombres propios porque PPMI favorece los eventos poco frecuentes. Finalmente reduje la matriz a 100 dimensiones con TruncatedSVD, que explican 24.56 por ciento de la varianza, y los vecinos de empresa fueron compañía, plataforma, asociación, tecnología y entidad. Las filas ya son vectores porque describen los contextos de cada palabra, PPMI corrige el dominio de la frecuencia y SVD comprime una matriz grande, dispersa y ruidosa.

Sección 3. Investigación

CBOW predice la palabra central a partir del promedio de su contexto y skip-gram predice cada palabra del contexto a partir de la central, por lo que skip-gram funciona mejor con palabras raras al generar varios ejemplos por aparición. Negative sampling evita calcular un softmax sobre todo el vocabulario y lo cambia por decidir si un par es real o falso usando $\sigma(v_w \cdot c_c)$, donde v_w es el vector de la palabra central y c_c el de la palabra de contexto. Por eso se aprenden dos tablas, una de palabras y otra de contextos, y al final se conserva la de palabras. Word2vec aprende de ventanas locales, GloVe ajusta los vectores a los conteos globales y fastText arma cada vector con n-gramas de caracteres, algo muy útil en español por su morfología. En gensim vector_size es la dimensión, window el tamaño de la ventana, min_count la frecuencia mínima, sg elige skip-gram o CBOW, negative es la cantidad de negativos, sample submuestrea palabras frecuentes y epochs son las pasadas por el corpus. En PyTorch nn.Embedding es una tabla con num_embeddings filas de tamaño embedding_dim, padding_idx deja una fila en ceros, from_pretrained con freeze decide si la tabla se entrena y nn.EmbeddingBag promedia o suma las palabras de un documento según mode. Multiplicar un one-hot por E selecciona una fila porque todos los demás términos se multiplican por cero, y en el notebook lo comprobé.

Sección 4. Variantes de Word2Vec y fastText

Entrené el modelo base con skip-gram, ventana 5, 100 dimensiones, min_count 2, negative 5 y 20 épocas, y cinco variantes cambiando un parámetro a la vez. El base tardó unos 15 segundos, CBOW unos 4, ventana 2 unos 8, ventana 10 unos 24, dimensión 50 unos 11 y dimensión 300 unos 29. La idea de que las ventanas pequeñas dan vecinos más sintácticos solo se vio un poco, porque con ventana 2 apareció mercados entre los vecinos de mercado, mientras que con ventana 10 los vecinos fueron más temáticos, como devops, computación y nube para tecnología. Con un corpus de este tamaño la evidencia es débil. Elegí el modelo base como mejor modelo porque dio vecinos coherentes para banco, gobierno y empresa, como central, agrario, popular y pichincha para banco, mientras que CBOW falló con banco y las ventanas de 2 y 10 fallaron con tecnología y gobierno.

El mismo modelo entrenado sobre el Quijote mostró que el significado viene del corpus. Gobierno pasó de tener vecinos como congreso y electo a ínsula, condado y gobierne, empresa pasó de startup y criptomoneda a aprieto y peligrosa, y casa pasó de teletrabajar y remoto a aldea y hacienda. Con fastText de gensim pude obtener vectores para palabras que nunca vio. El error gobeirno quedó cerca de gobierno y gobernar, emprendiéramos cerca de emprender y emprendedor, y la palabra inventada criptoempresa cerca de criptomoneda y empresa. Word2Vec en cambio dio KeyError en las tres, porque solo guarda un vector por palabra completa.

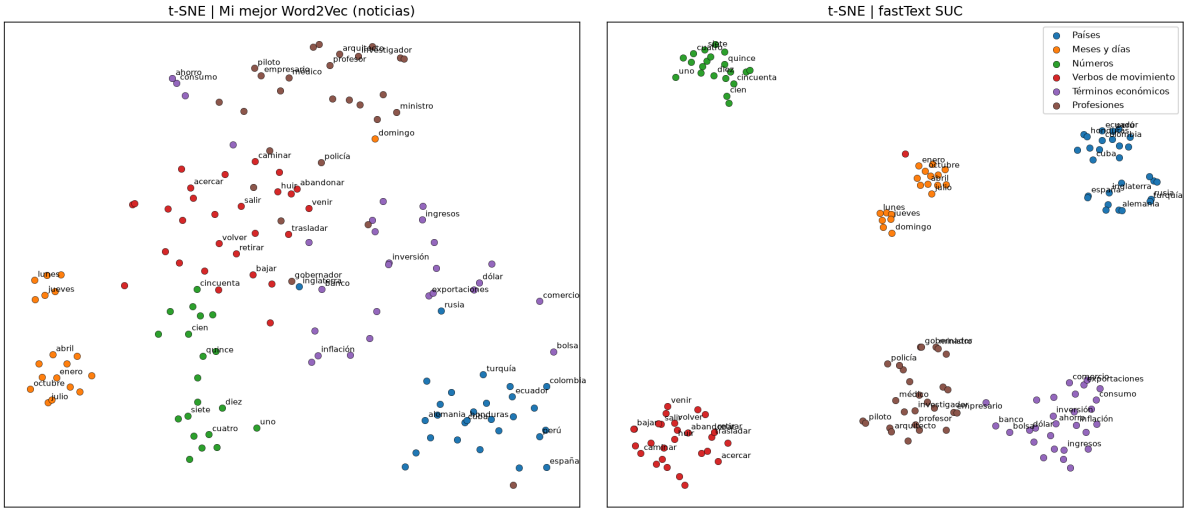
Sección 5. Similitud, vecinos y analogías

Comparé cuatro modelos, SVD sobre PPMI, mi Word2Vec de noticias, el Word2Vec del Quijote y fastText SUC. Mi función de coseno dio exactamente lo mismo que gensim. Los vecinos de SUC fueron los más razonables, por ejemplo mansión, casita y cabaña para casa. Los antónimos salieron con similitudes altas, como subir y bajar con 0.9372 en SUC y 0.6319 en mi modelo, cuando el coseno promedio entre palabras al azar es de 0.2059 y 0.3723. Esto pasa porque los antónimos aparecen en los mismos contextos, así que similitud distribucional no es sinonimia.

Con mis 20 pares calificados de 0 a 10 la correlación de Spearman fue de 0.5235 para SUC, 0.1055 para mi modelo de noticias, menos 0.1511 para SVD con 9 pares fuera del vocabulario y menos 0.1429 para el Quijote con 12 pares fuera. SUC fue el único significativo. En las 19 analogías SUC acertó 16 en la primera posición y las 19 entre las primeras cinco, con aciertos completos en género, gentilicios, plurales y adjetivos. Mi modelo cubrió 14 analogías y no acertó ninguna en la primera posición. Al repetir cinco analogías sin excluir las palabras de la consulta, en 10 de 16 casos la primera respuesta fue una palabra de la propia consulta, lo que muestra que rey menos hombre más mujer no es igual a reina, solo queda cerca.

Sección 6. Visualización del espacio

Proyecté 150 palabras de seis grupos con PCA y t-SNE. En SUC los grupos se separan claramente, mientras que en mi modelo se forman países, meses y números pero los verbos de movimiento y los términos económicos quedan mezclados. La silueta en t-SNE fue de 0.7223 para SUC y 0.3359 para mi modelo. En t-SNE se puede confiar en los grupos porque conserva los vecinos cercanos, pero no en la distancia entre grupos porque solo optimiza distancias locales.



Sección 8. Clasificación, búsqueda y sesgos

Esta sección no aparece en el PDF, pero la piden la sección 3 y los entregables, por lo que la inferí. Usé la misma partición del Laboratorio 3 y el baseline de TF-IDF con regresión logística del Laboratorio 4. Los embeddings se usaron con nn.EmbeddingBag en modo promedio, congelados o con fine-tuning. Los resultados de prueba fueron los siguientes.

Modelo	**100% accuracy**	**100% macro-F1**	**10% macro-F1**
TF-IDF y regresión logística	0.7953	0.7048	0.2462
Mi Word2Vec congelado	0.8246	0.8049	0.5854
Mi Word2Vec con fine-tuning	0.8655	0.8642	0.6332
SUC congelado	0.7895	0.7521	0.4905
SUC con fine-tuning	0.8596	0.8184	0.6107

En la búsqueda semántica el promedio de vectores y TF-IDF tuvieron una precisión parecida en los primeros 10 resultados, aunque el promedio de vectores ganó en la consulta de innovación con 0.7 contra 0.4. En sesgos medí el coseno de 24 profesiones con hombre y con mujer. En SUC 22 de 24 profesiones quedaron más cerca de hombre, con soldado y maestro como las más inclinadas, y la analogía de médico para mujer devolvió obstetricia y ginecóloga. En mi modelo la medida no es confiable porque hombre aparece solo 7 veces en el corpus.

Análisis

Los vecinos de SVD sobre PPMI casi no se parecen a los de word2vec. SVD es simple y determinista pero solo cubre 2,000 palabras, mientras que word2vec cubre todo el vocabulario y aprende de palabras raras. GloVe une ambos enfoques porque entrena vectores para reproducir los conteos globales. Mi Word2Vec quedó muy lejos de SUC en vecinos, similitud y analogías, y creo que la mayor parte de la diferencia se debe al tamaño del corpus, ya que SUC se entrenó con más de 4,000 veces más texto, aunque no hice un experimento que separe el efecto del método. La aritmética vectorial funcionó mejor en género, gentilicios, plurales y adjetivos, y fue más débil en capitales y tiempos verbales.

Al comparar noticias con el Quijote se ve el mismo problema que con el tagger del Laboratorio 6, donde el mismo modelo tuvo un F1 de 0.880 en Dracula y de 0.643 en Pride and Prejudice. Un modelo entrenado en un dominio o una época cambia de vocabulario y de significados al pasar a otro. En clasificación los embeddings sí superaron a TF-IDF, sobre todo con el 10 por ciento de los datos, donde TF-IDF cayó a 0.2462 de macro-F1. El fine-tuning ayudó más con SUC porque adapta vectores generales al dominio, y con pocos datos congelar es una opción razonable porque la diferencia fue pequeña. Hay que tomar en cuenta que mi Word2Vec vio el texto de prueba al entrenarse, aunque sin etiquetas.

La palabra banco tiene un solo vector en cada modelo. En las noticias y en SUC sus vecinos son de bancos financieros, mientras que en el Quijote aparecen palabras como ristre y empuñadura, así que el vector mezcla todos los sentidos según el texto. Modelos contextuales como BERT resuelven esto al calcular un vector distinto para cada aparición según la oración. Por último, antes de usar embeddings preentrenados en un sistema que decide sobre personas auditaría sesgos de género, nacionalidad, raza y edad, revisaría la fuente del corpus, mediría el desempeño por grupo y mantendría revisión humana en las decisiones importantes.

Repositorio en GitHub: https://github.com/luispegr25/NLP